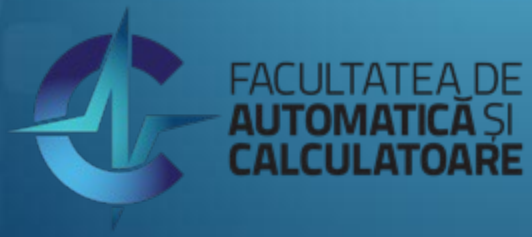


Algoritmi Paraleli și Distribuiți
Pthread + Primitive sincronizare

Radu-Ioan Ciobanu
radu.ciobanu@upb.ro





POSIX threads

```
pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);

void * threadFunction(void* arg)
{

}

pthread_join(thread, NULL);
```

Compilare pthread

gcc –o executabil cod.c –lpthread –lrt

#include<pthread.h>

#include<semaphore.h>

Pthread

```
pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);


void * threadFunction(void* arg)
{

}

pthread_join(thread, NULL);
```

Acest element reprezintă thread-ul.
Poate fi considerat handle

Pthread

```
pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);

void * threadFunction(void* arg)
{
}

pthread_join(thread, NULL);
```

Aici am putea să facem recomandări sistemului de operare(ex., să folosească anumite core-uri).

Pthread

```
pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);

void * threadFunction(void* arg)
{
}

pthread_join(thread, NULL);
```

Când se crează thread-ul, acesta va începe la această funcție.

Pthread

```
pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);

void * threadFunction(void* arg)
{
        Așa trimitem date thread-ului
}

pthread_join(thread, NULL);
```

Pthread

```
pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);

void * threadFunction(void* arg)
{
        Astfel se pot extrage
        informații din thread
}

pthread_join(thread, NULL);
```

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

Pthread



```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```



Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```



Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```



Pthread

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

Pthread

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```



```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
     pthread_join(tid[i], NULL);
   }
   return 0;
}
```

Pthread

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
     pthread_join(tid[i], NULL);
   }
   return 0;
}
```

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```

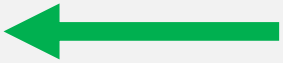
Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```



```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```





```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```

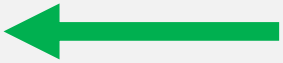
Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```



```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
}
```





```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
}
```

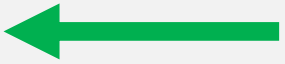
Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```



```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```



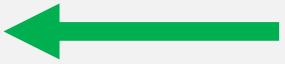
Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
}
```



```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
}
```



Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
}
```



```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
}
```



Pthread

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
     pthread_join(tid[i], NULL);
   }
   return 0;
}
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```

Pthread

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
     pthread_join(tid[i], NULL);
   }
   return 0;
}
```

Pthread

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```



```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
     pthread_join(tid[i], NULL);
   }
   return 0;
}
```



Pthread

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```



```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
     pthread_join(tid[i], NULL);
   }
   return 0;
}
```



Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```



Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```



Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```



Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```



Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```



Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```



Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```



ÎN MAIN

Înainte de a porni thread-urile

pthread_mutex_t **mutex**;

pthread_mutex_init(&**mutex**, NULL);

Mutex Pthread

```
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);
```

Poate fi folosit să anunțe că acest mutex e împărțit mai multor procese

Mutex Pthread

pthread_mutex_lock(&**mutex**);

load(a, eax)

eax = eax + 2

write(a, eax)

pthread_mutex_unlock(&**mutex**);

ÎN MAIN

După ce au terminat thread-urile

pthread_mutex_destroy(&**mutex**);

Semaphore

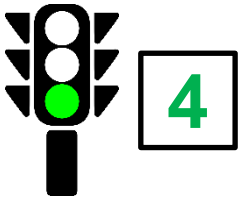
ÎN MAIN

Înainte de a porni thread-urile

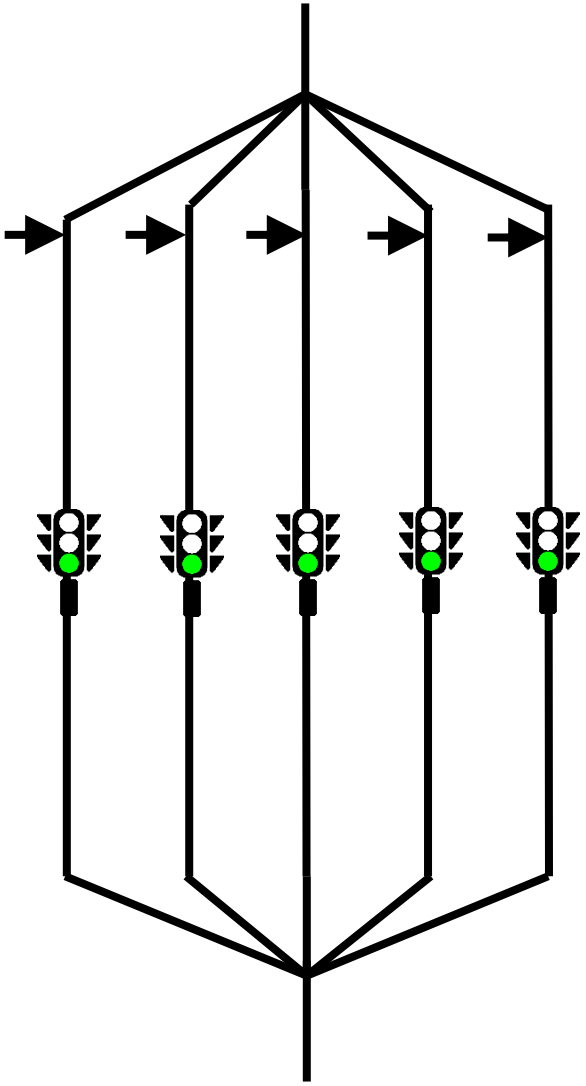
```
sem_t semaphore;
int semaphore_value= 4;
sem_init(& semaphore, 0, semaphore_value);
```

Semaphore

```
sem_wait(&semaphore);
```

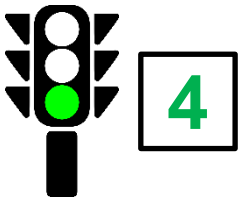


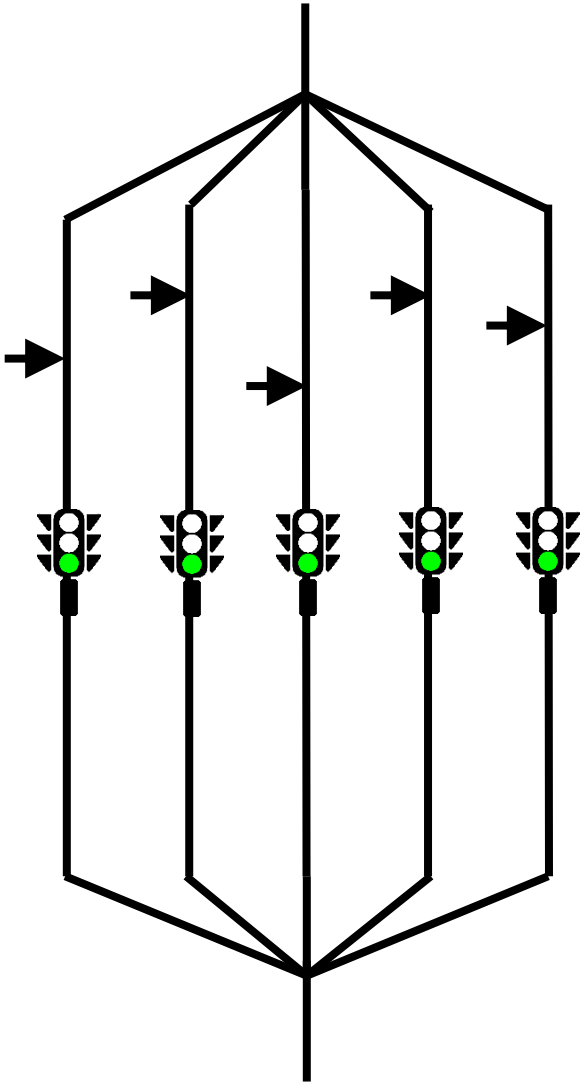
P() sau Proberen
 - Dijsktra



Semaphore

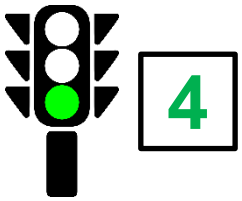
```
sem_wait(&semaphore);
```

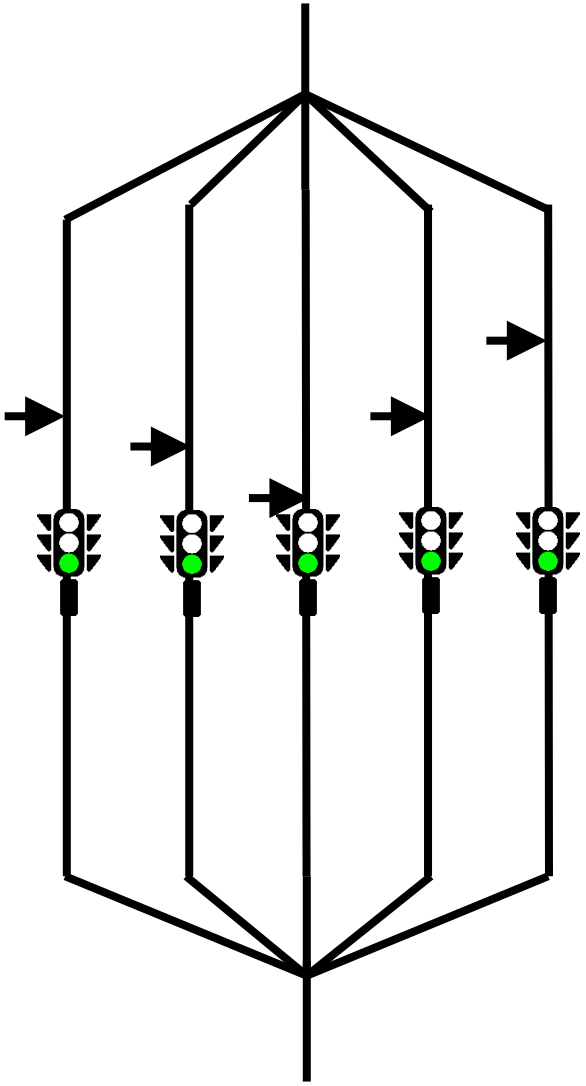




Semaphore

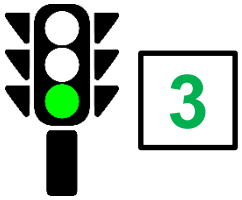
```
sem_wait(&semaphore);
```

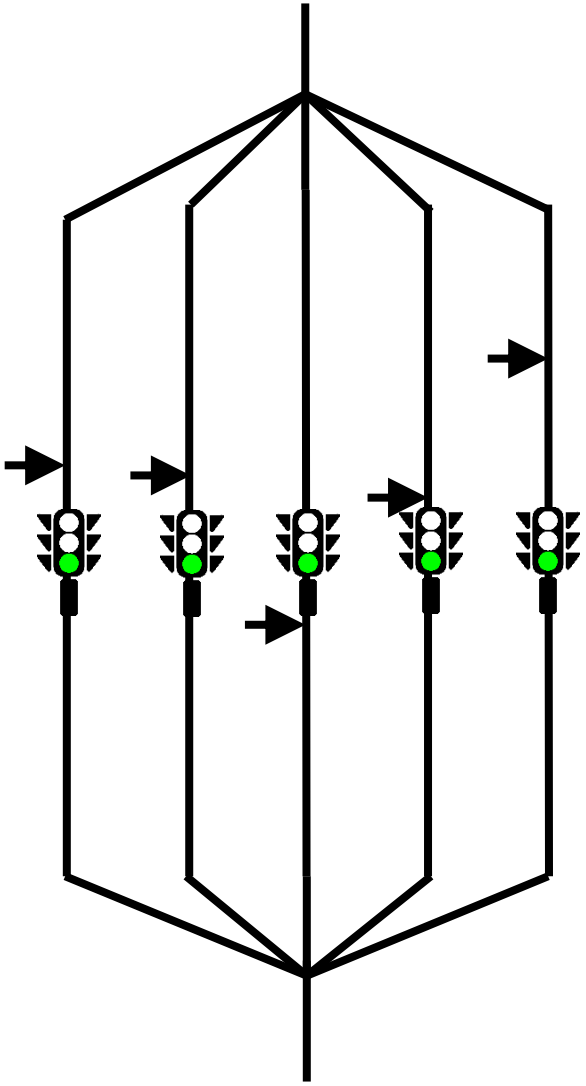




Semaphore

```
sem_wait(&semaphore);
```

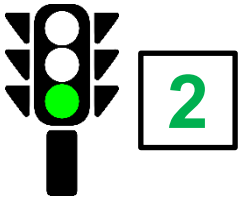


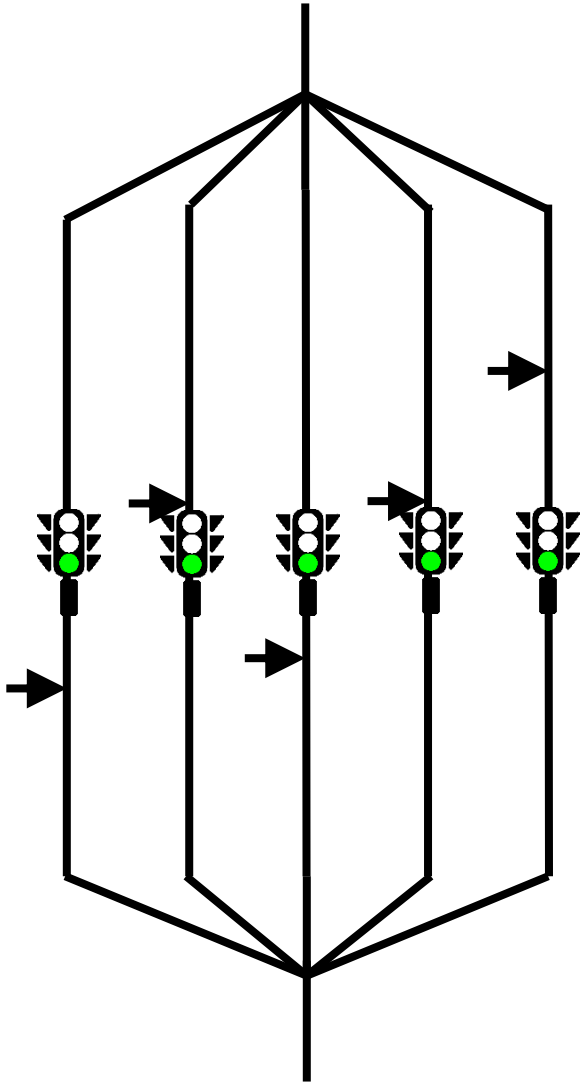


Semaphore

Nu contează că doua thread-uri au ajuns simultan la semafor, acesta este protejat, la fel ca un mutex.

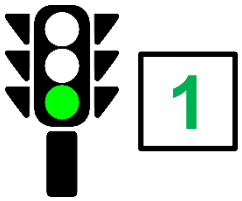
```
sem_wait(&semaphore);
```

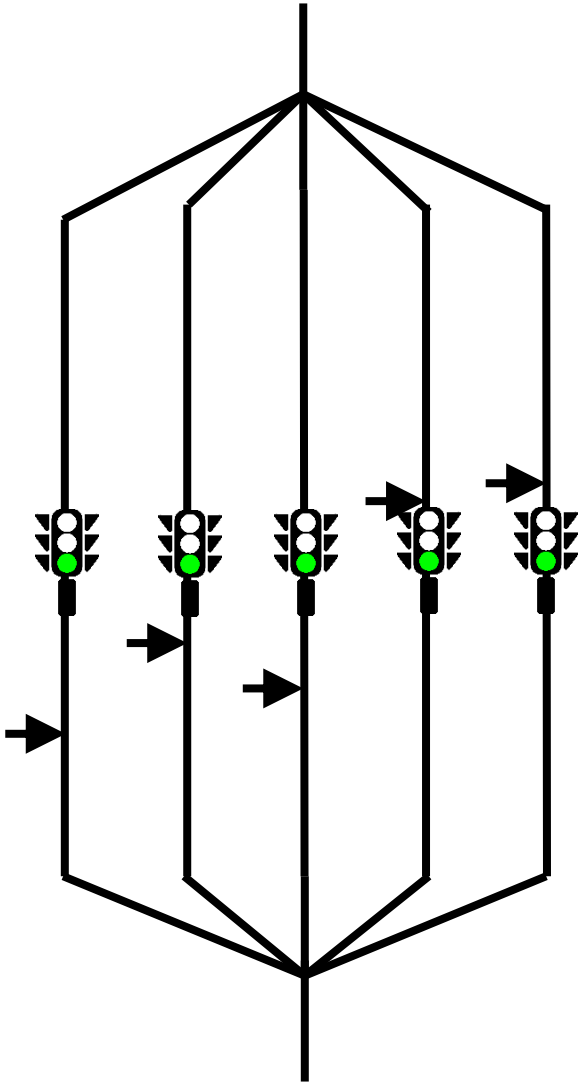




Semaphore

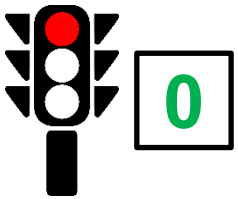
```
sem_wait(&semaphore);
```

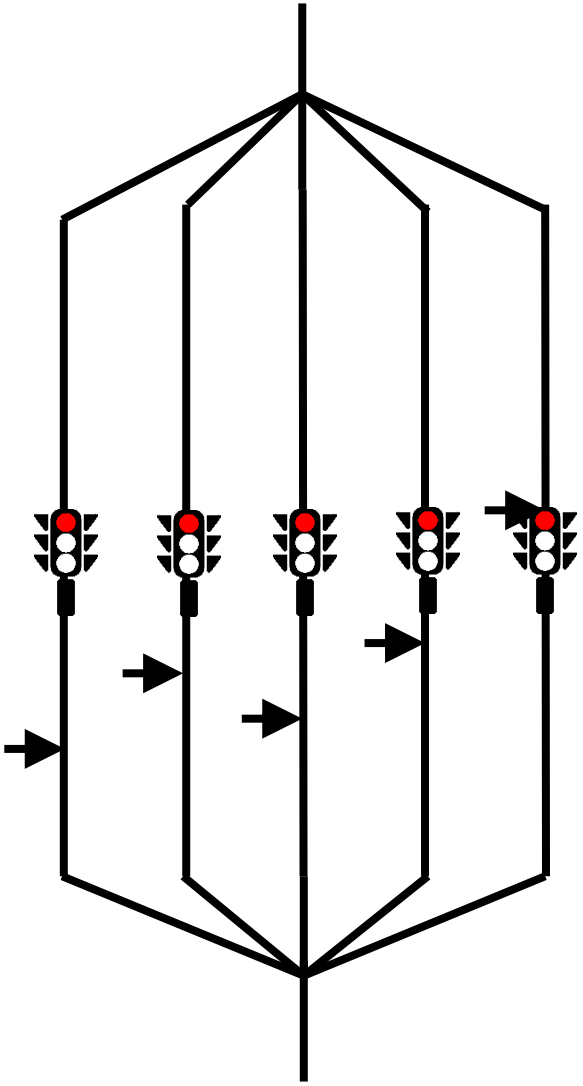




Semaphore

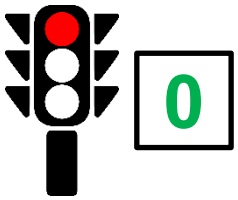
```
sem_wait(&semaphore);
```

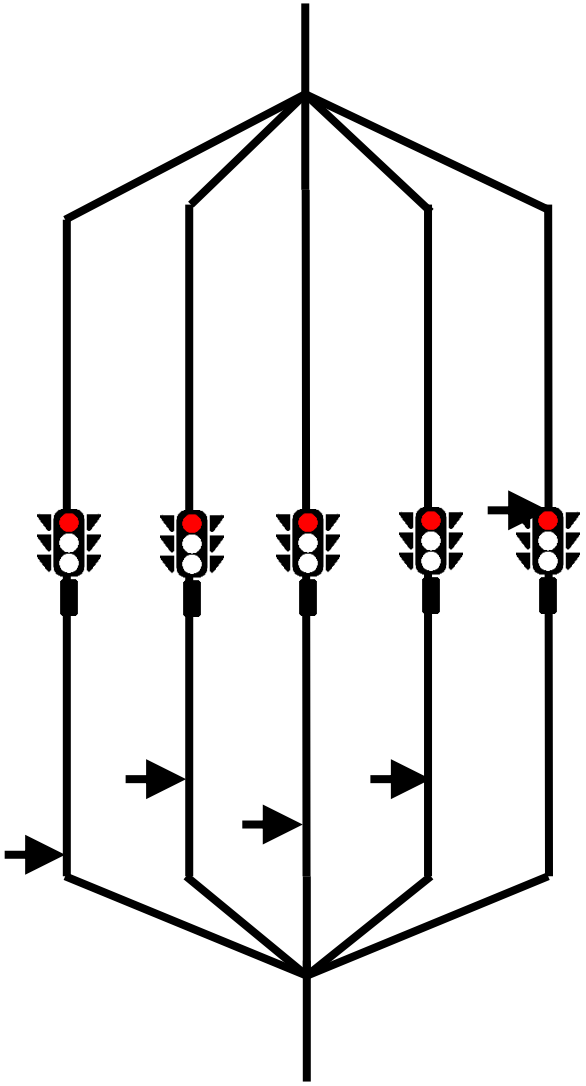




Semaphore

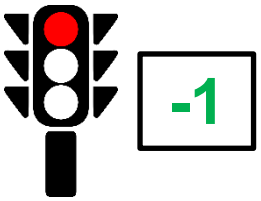
```
sem_wait(&semaphore);
```

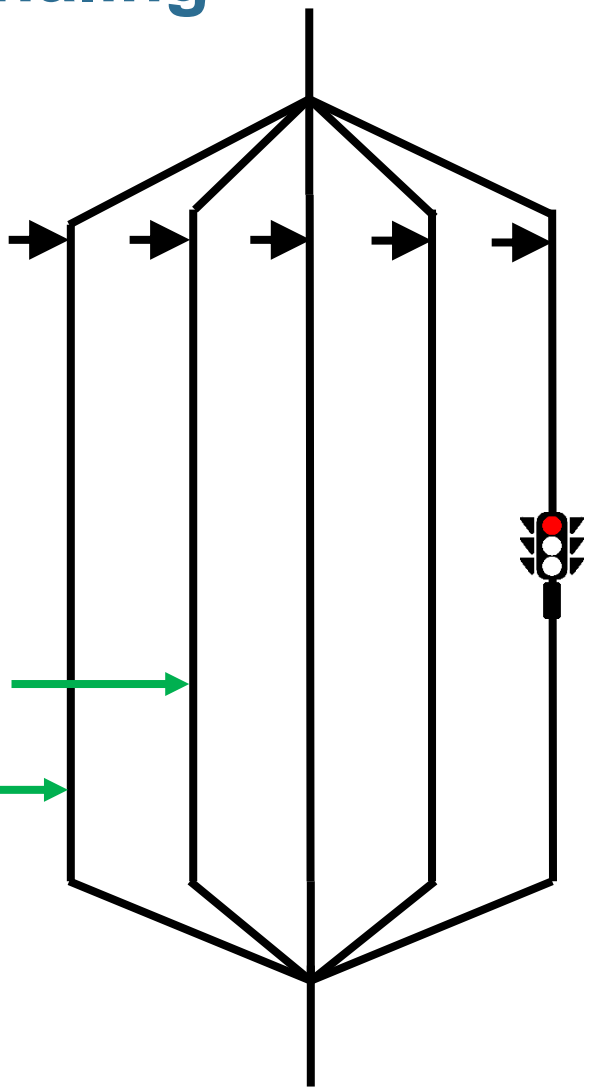




Semaphore – Signaling

```
sem_wait(&semaphore);
```





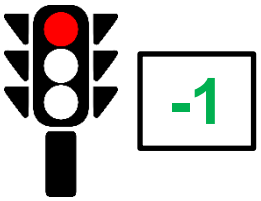
```
sem_post(&semaphore);
```

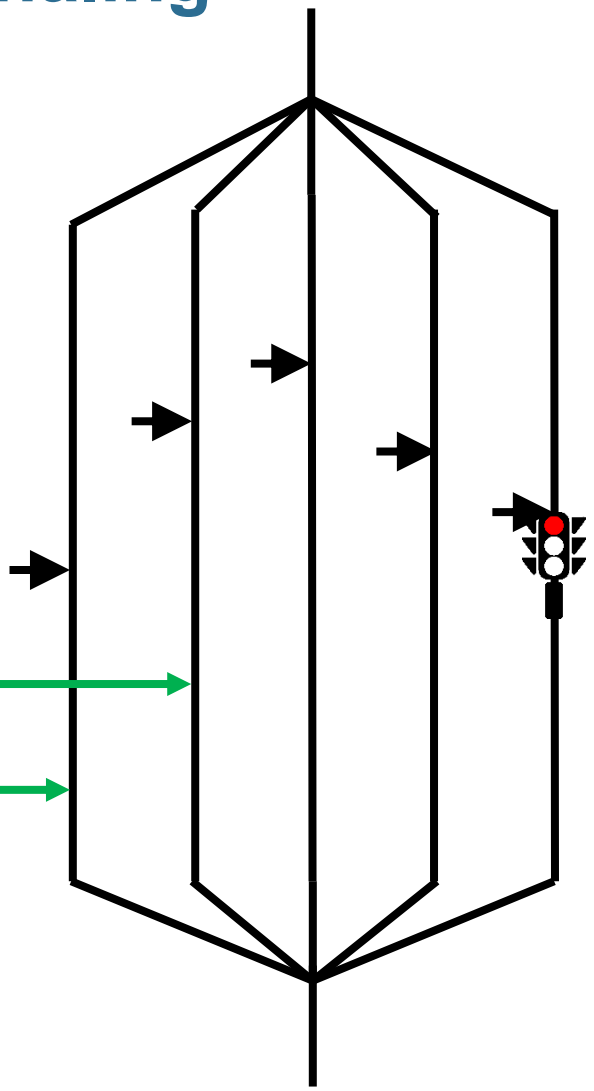
```
sem_post(&semaphore);
```

V() sau Verogen
- Dijsktra

Semaphore – Signaling

```
sem_wait(&semaphore);
```



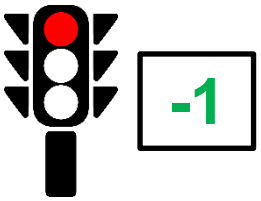


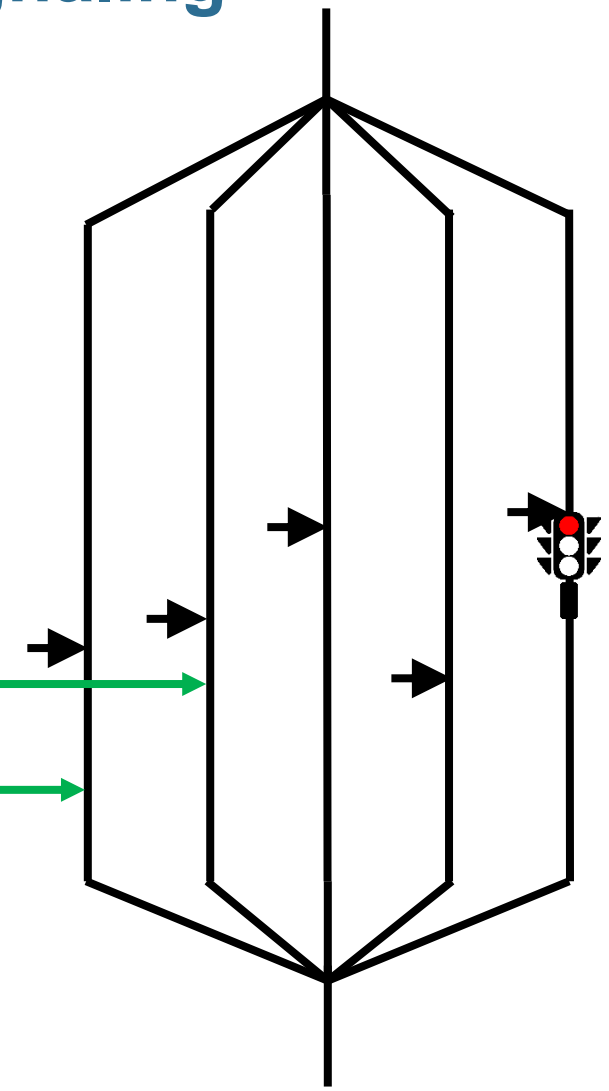
```
sem_post(&semaphore);
```

```
sem_post(&semaphore);
```

Semaphore – Signaling

```
sem_wait(&semaphore);
```



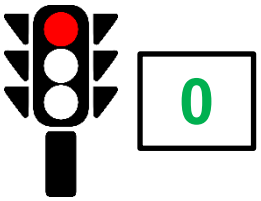


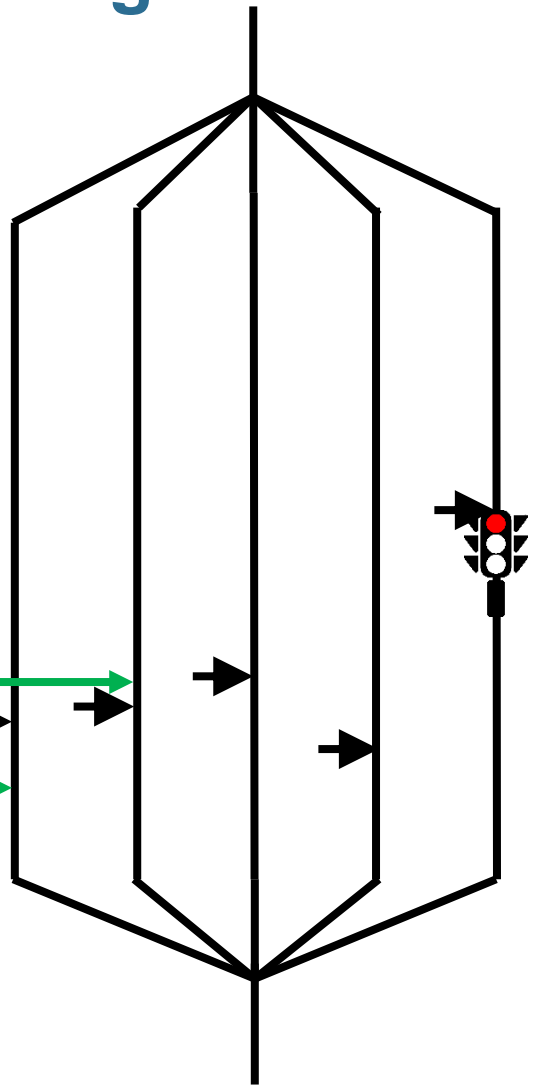
```
sem_post(&semaphore);
```

```
sem_post(&semaphore);
```

Semaphore – Signaling

```
sem_wait(&semaphore);
```

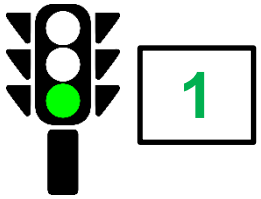


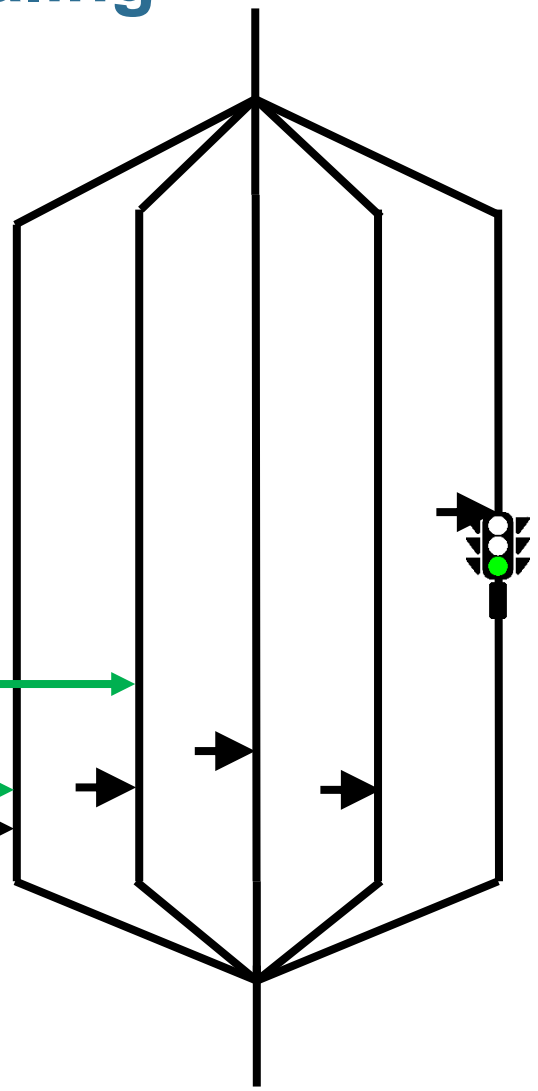


```
sem_post(&semaphore);
sem_post(&semaphore);
```

Semaphore – Signaling

```
sem_wait(&semaphore);
```



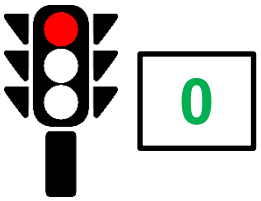


```
sem_post(&semaphore);
```

```
sem_post(&semaphore);
```

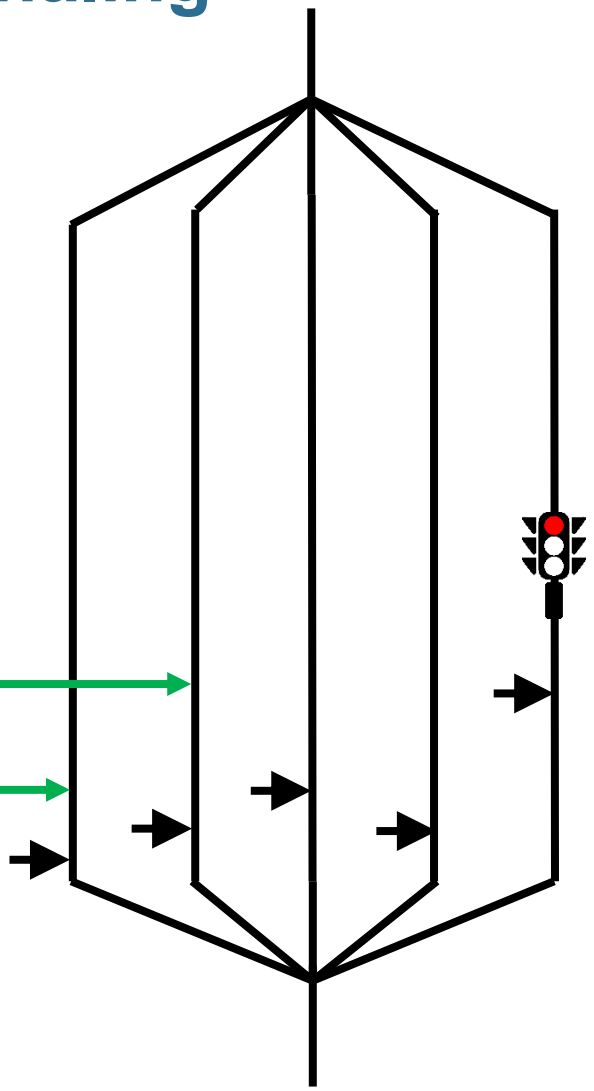
Semaphore – Signaling

```
sem_wait(&semaphore);
```




```
sem_post(&semaphore);
```

```
sem_post(&semaphore);
```



Semaphore

ÎN MAIN

După ce au terminat thread-urile

sem_destroy(& **semaphore**);

Barrier



ÎN MAIN

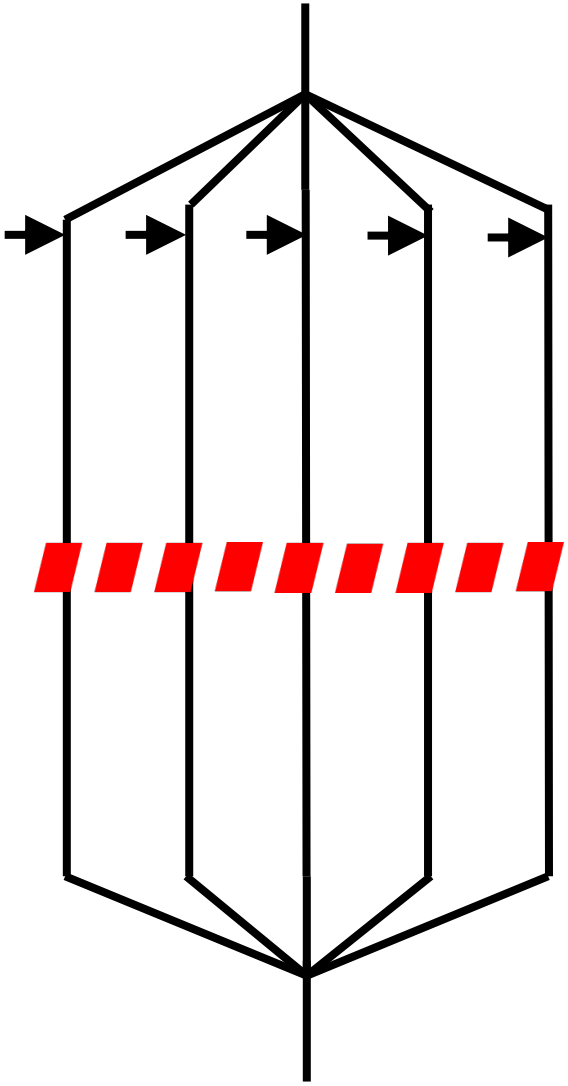
Înainte de a porni thread-urile

```
pthread_barrier_t barrier;
int num_threads = 5;
pthread_barrier_init(&barrier, NULL, num_threads);
```

Barrier

pthread_barrier_wait(&**barrier**);

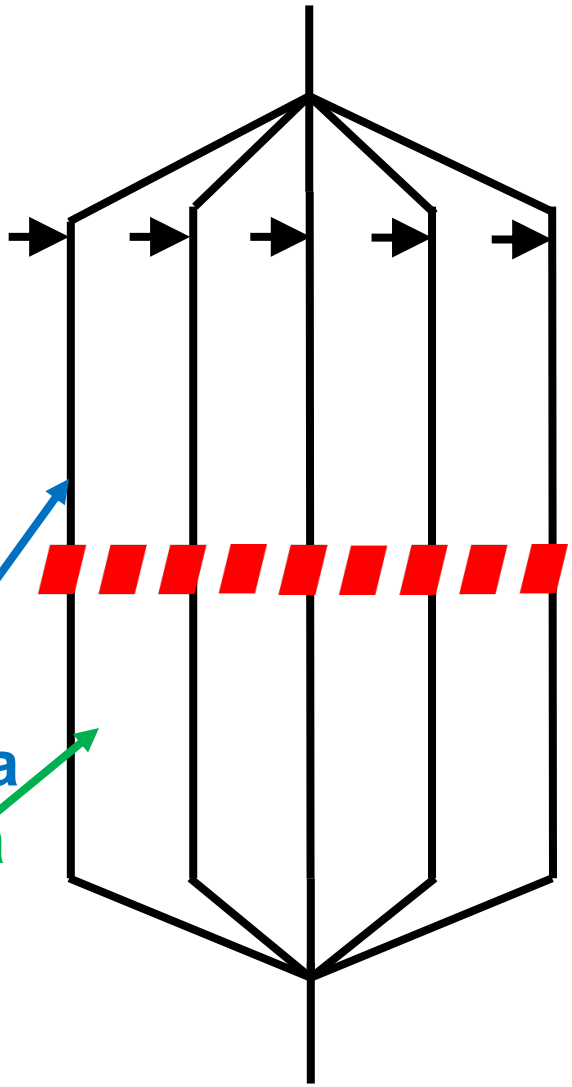
5



Barrier

pthread_barrier_wait(&barrier);

5

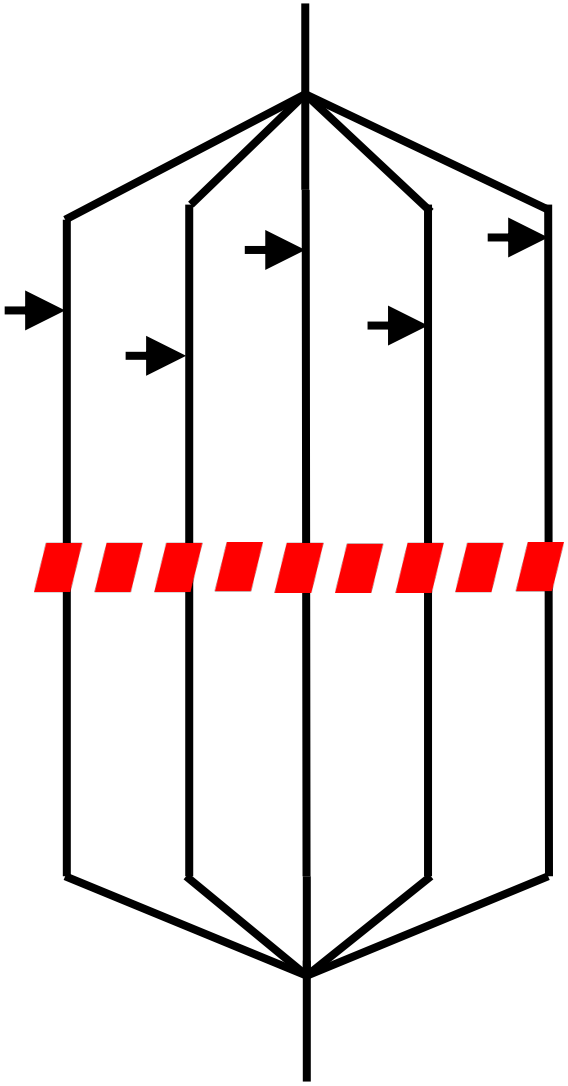


Pentru toate thread-urile codul acesta
Este executat înainte de codul acesta

Barrier

pthread_barrier_wait(&**barrier**);

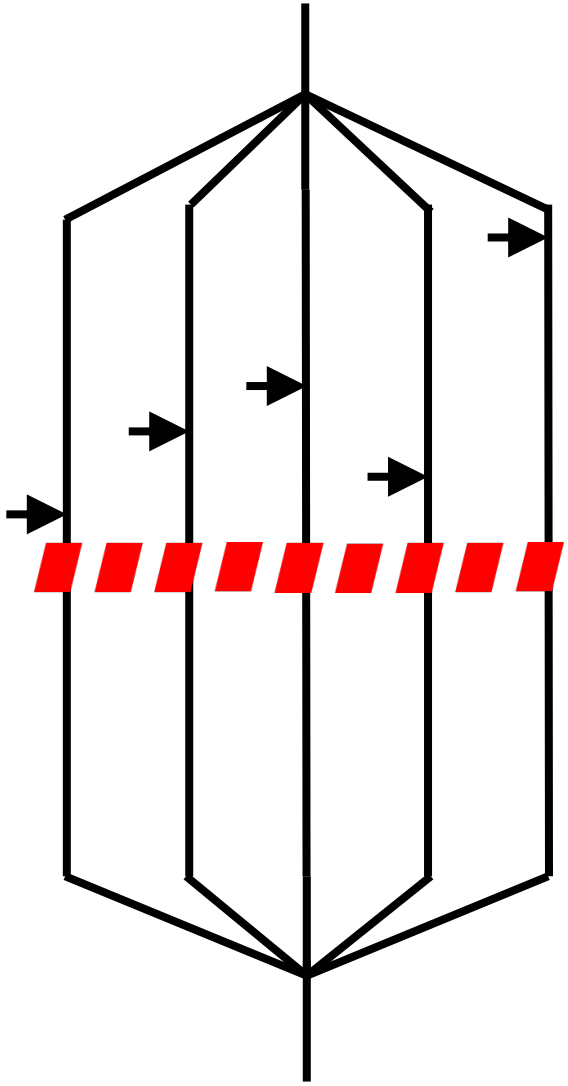
5



Barrier

pthread_barrier_wait(&**barrier**);

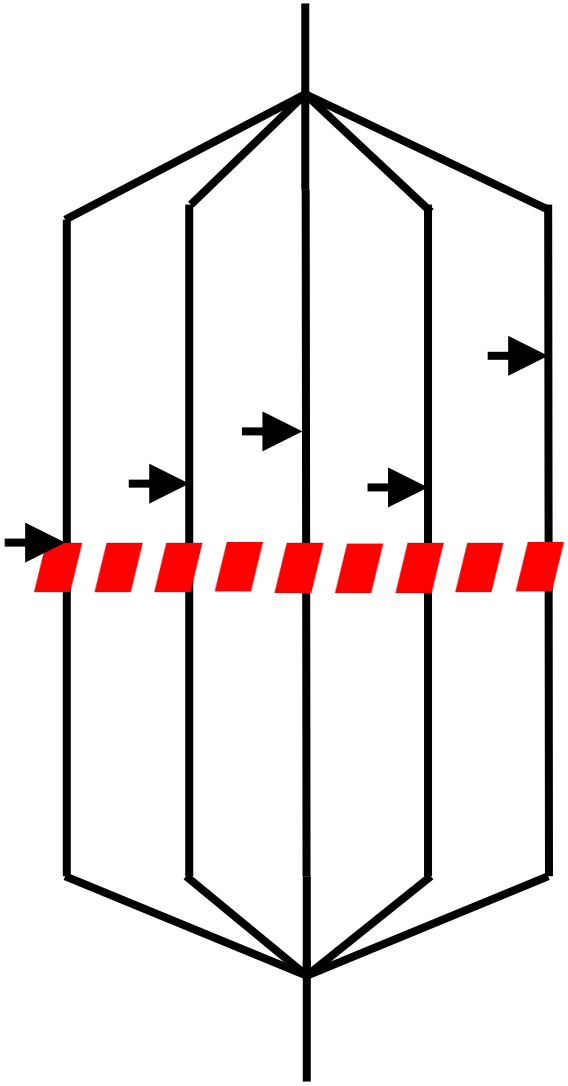
5



Barrier

pthread_barrier_wait(&**barrier**);

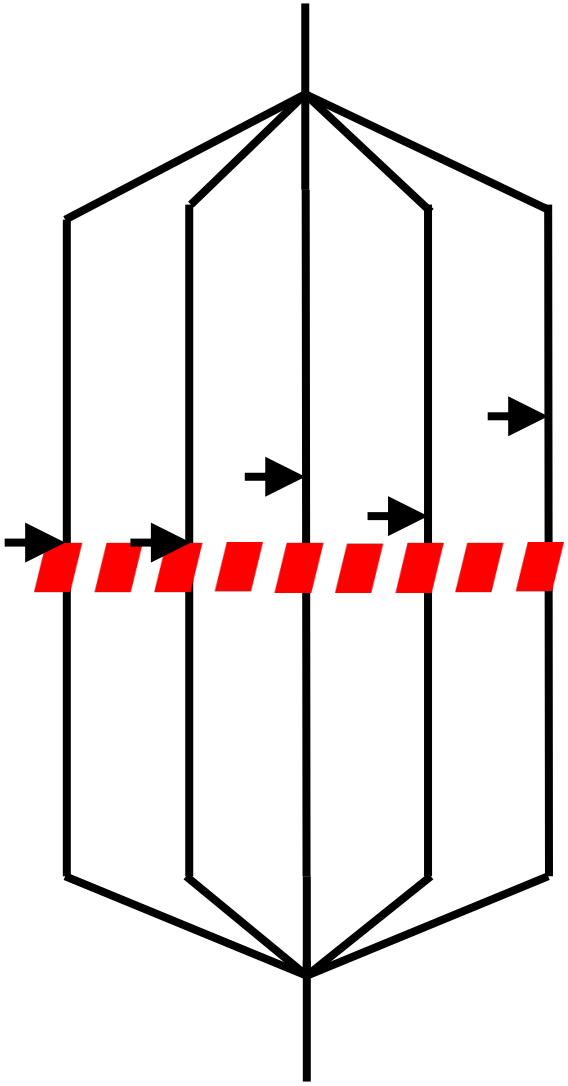
5



Barrier

pthread_barrier_wait(&**barrier**);

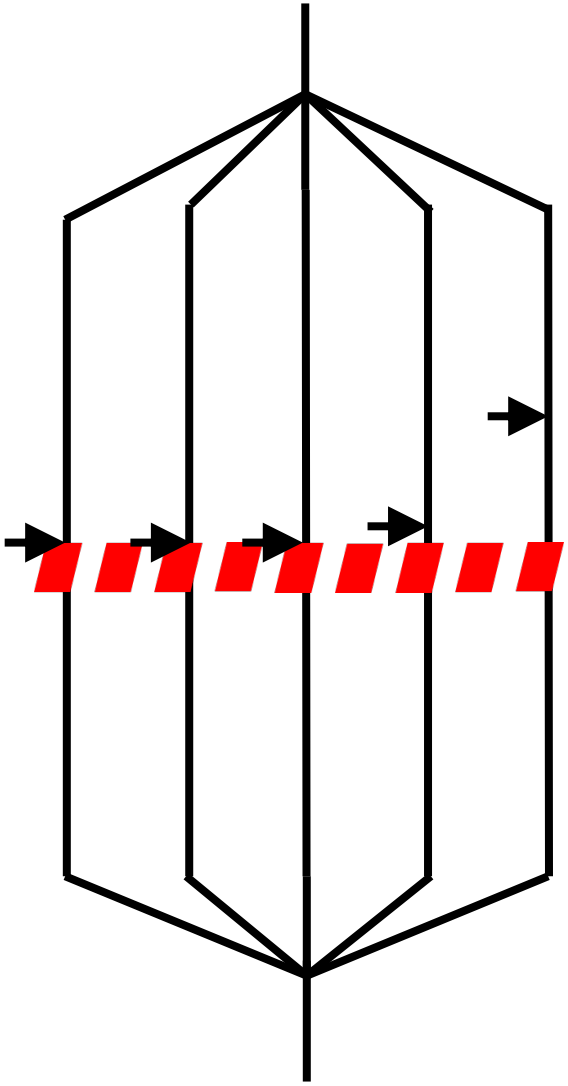
3



Barrier

pthread_barrier_wait(&**barrier**);

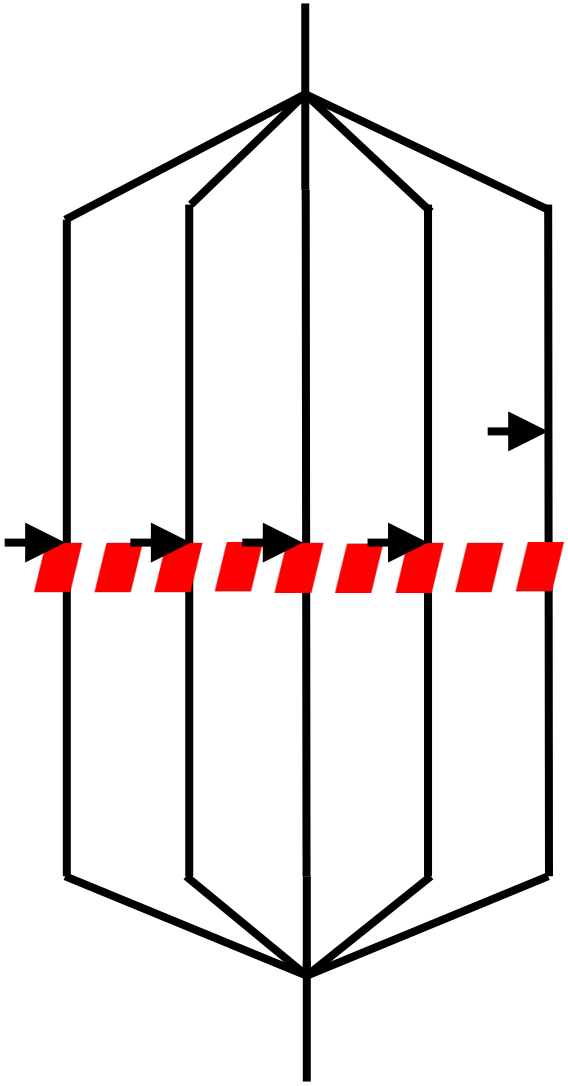
2



Barrier

pthread_barrier_wait(&**barrier**);

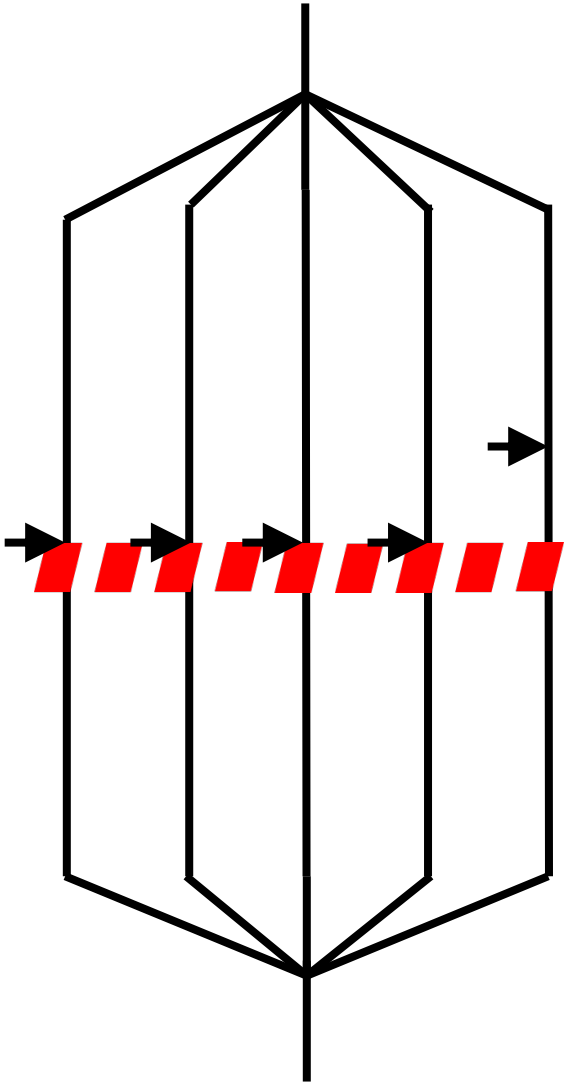
1



Barrier

pthread_barrier_wait(&**barrier**);

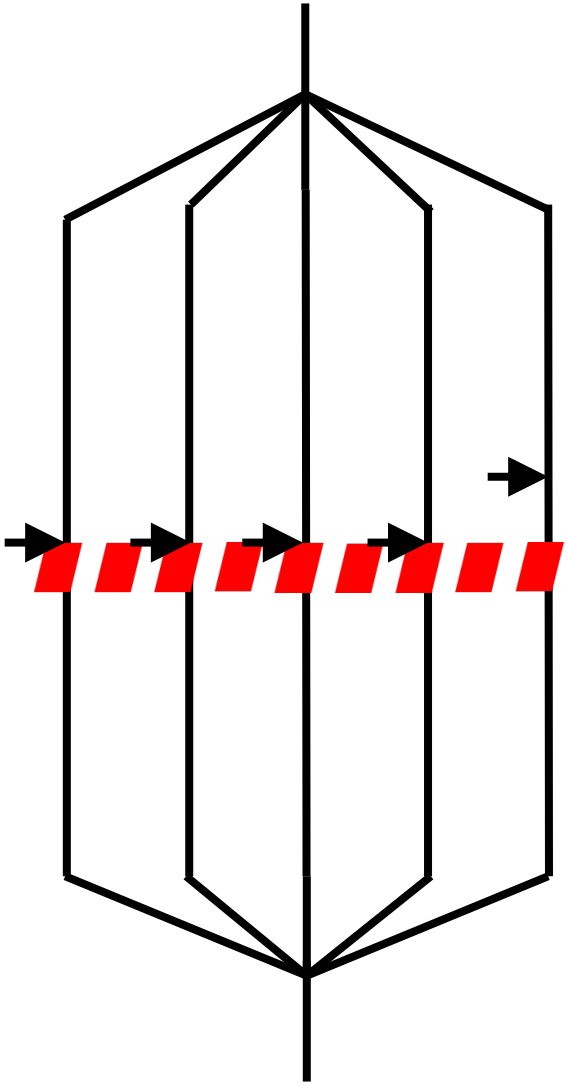
1



Barrier

pthread_barrier_wait(&**barrier**);

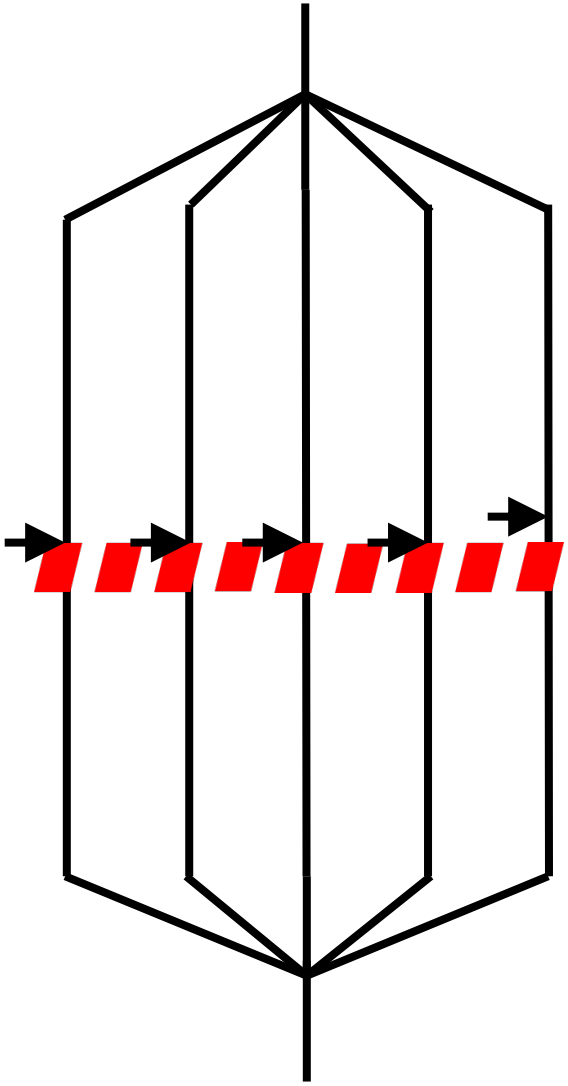
1



Barrier

pthread_barrier_wait(&**barrier**);

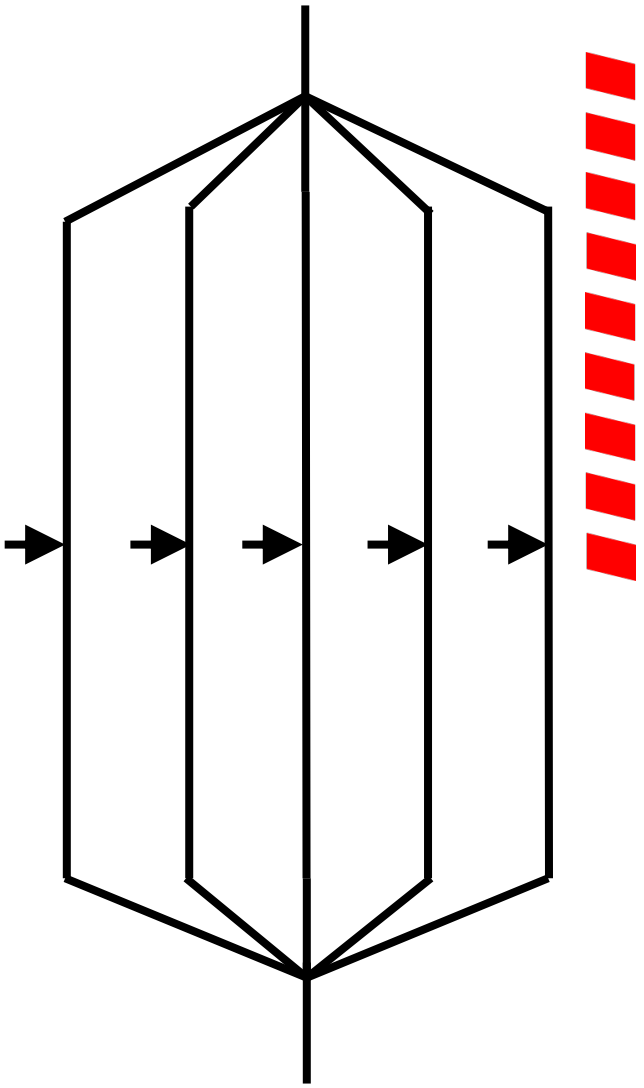
1



Barrier

pthread_barrier_wait(&**barrier**);

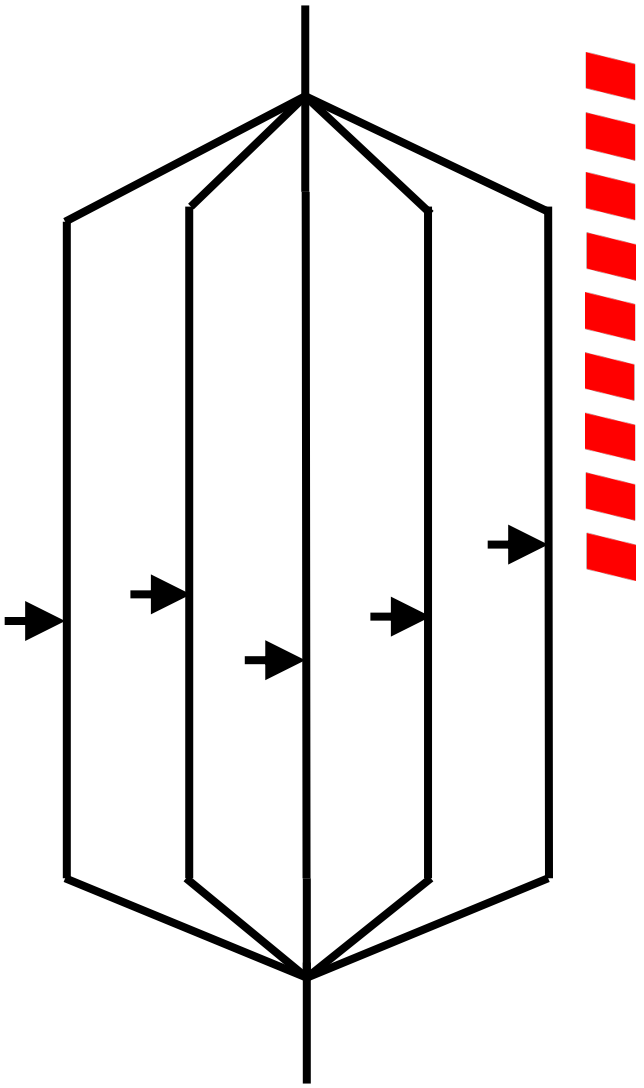
0



Barrier

pthread_barrier_wait(&**barrier**);

0



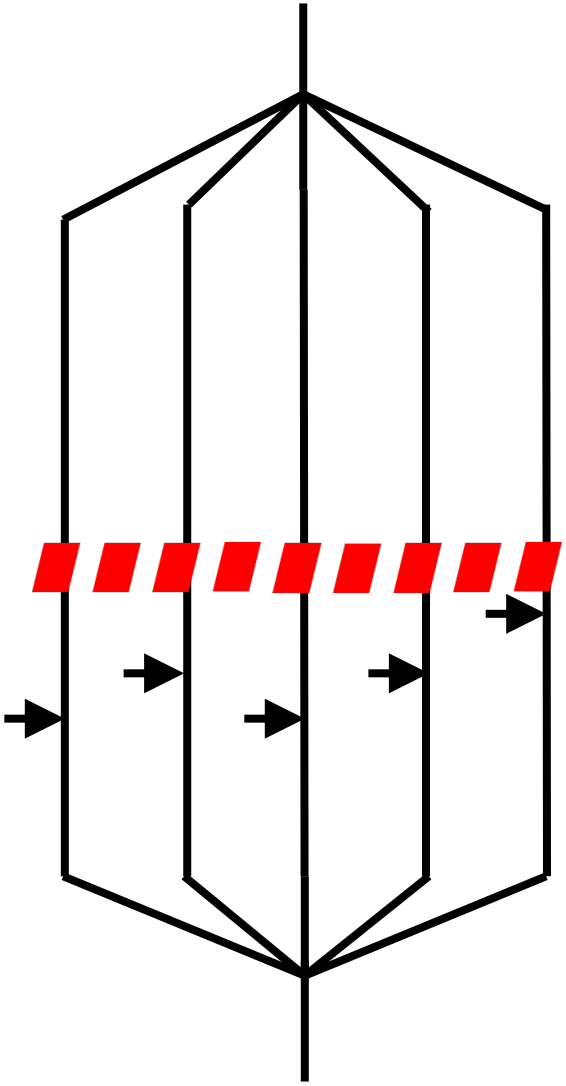
Barrier

Cum știe o barieră când să se reseteze?

O soluție ar fi:
Reusable Barrier in
The Little Book of Semaphores
By Allen B. Downey

pthread_barrier_wait(&barrier);

5



ÎN MAIN

După ce au terminat thread-urile

```
pthread_barrier_destroy(&barrier);
```

Unele probleme nu au soluție paralelă

Calculating the hash of a hash of a hash …

of a string.

Deep First Search

Huffman decoding

Outer loops of most simulations

P complete problems

Paralelizarea prin împărțirea muncii

Sunt o serie de probleme care sunt extrem de ușor paralelizabile.

Embarrassingly parallel

Probleme ”Embarrassingly parallel”

Multiplicare unui vector cu un scalar

9	6	9	4	2	7	6	5	6	1

* 3

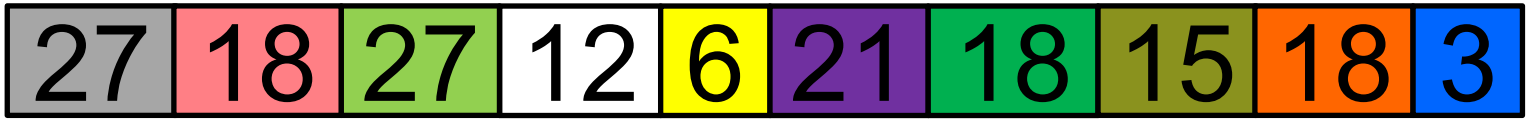
27	18	27	12	6	21	18	15	18	3

Probleme ”Embarrassingly parallel”

Toate calculele pot fi efectuate în același timp



* 3



Probleme ”Embarrassingly parallel”

Câte elemente sunt?



Probleme ”Embarrassingly parallel”

Câte elemente sunt?



Probleme ”Embarrassingly parallel”

Câte elemente sunt? **N**



Probleme ”Embarrassingly parallel”

Dar câte elemente de procesare?



Probleme ”Embarrassingly parallel”

Dar câte elemente de procesare? **P**



Probleme ”Embarrassingly parallel”

Dar câte thread-uri?



Probleme ”Embarrassingly parallel”

Dar câte thread-uri? **P**



În majoritatea cazurilor obținem performanță maximă când numărul de thread-uri este egal cu numărul de elemente de procesare, sau core-uri.

Probleme ”Embarrassingly parallel”

Cum este P față de N?



Probleme ”Embarrassingly parallel”

$$P << N$$



Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?



Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?



Thread 1

Thread 2

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?



Thread 1

Thread 2

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?



Thread 1

Thread 2

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim? Putem și random



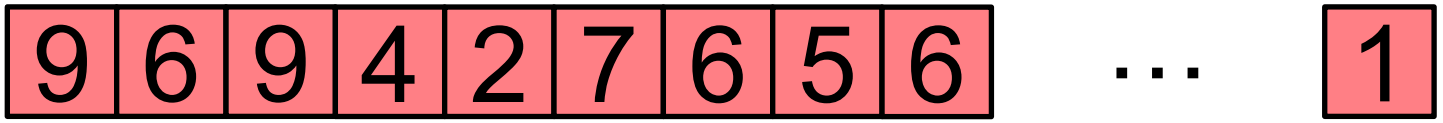
Thread 1

Thread 2

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?



Thread 1

Thread 2

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?



Este utilă?

Thread 1

Thread 2

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?



Ce ne dorim?

Thread 1

Thread 2

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?



Ce ne dorim?

Thread 1

Thread 2

Aproximativ același număr elemente

Probleme ”Embarrassingly parallel”

Aproximativ N/P elemente



Thread 1

Thread 2

Probleme ”Embarrassingly parallel”

Aproximativ N/P elemente



Dacă N nu se divide perfect la P?

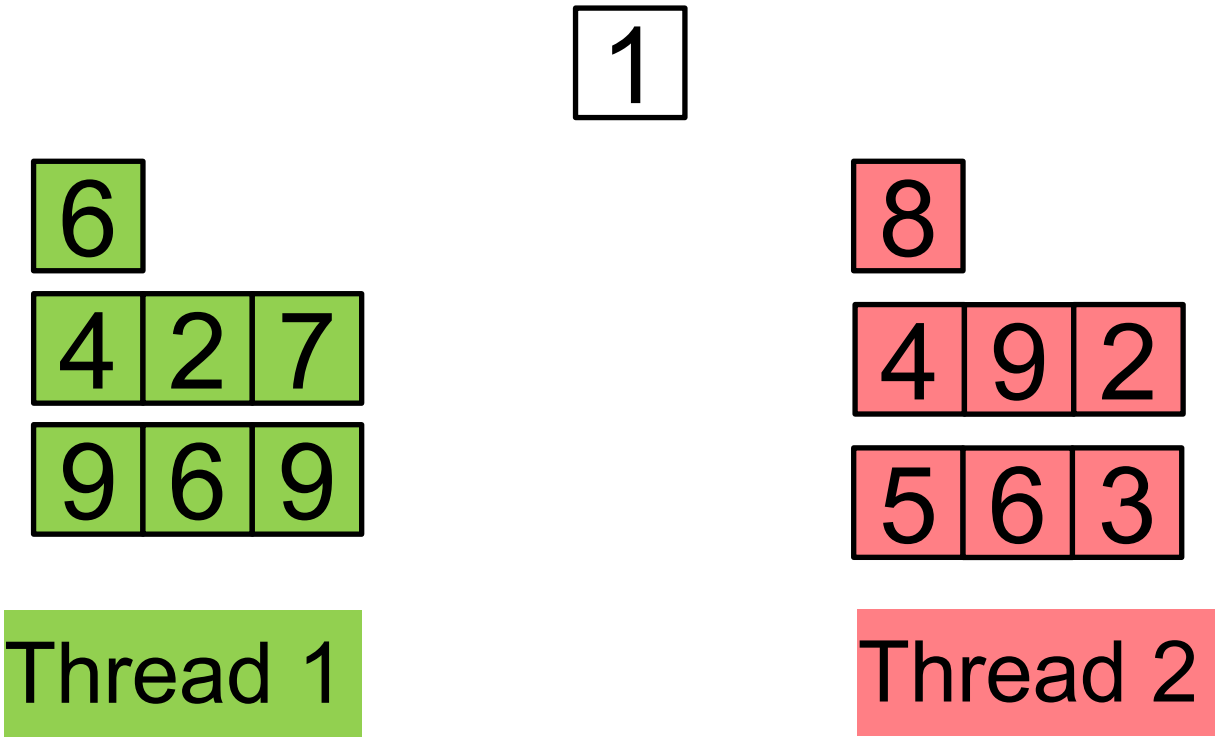
Thread 1

Thread 2

Probleme ”Embarrassingly parallel”

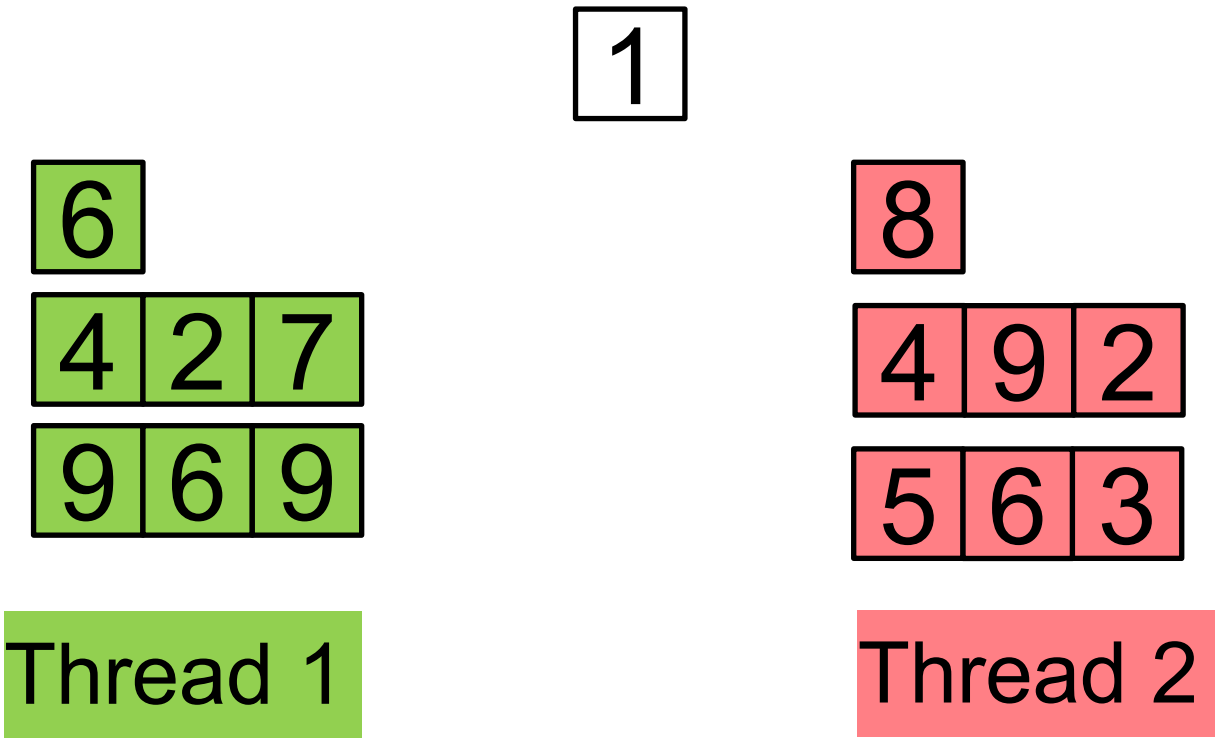
Aproximativ N/P elemente

Dacă N nu se divide perfect la P?



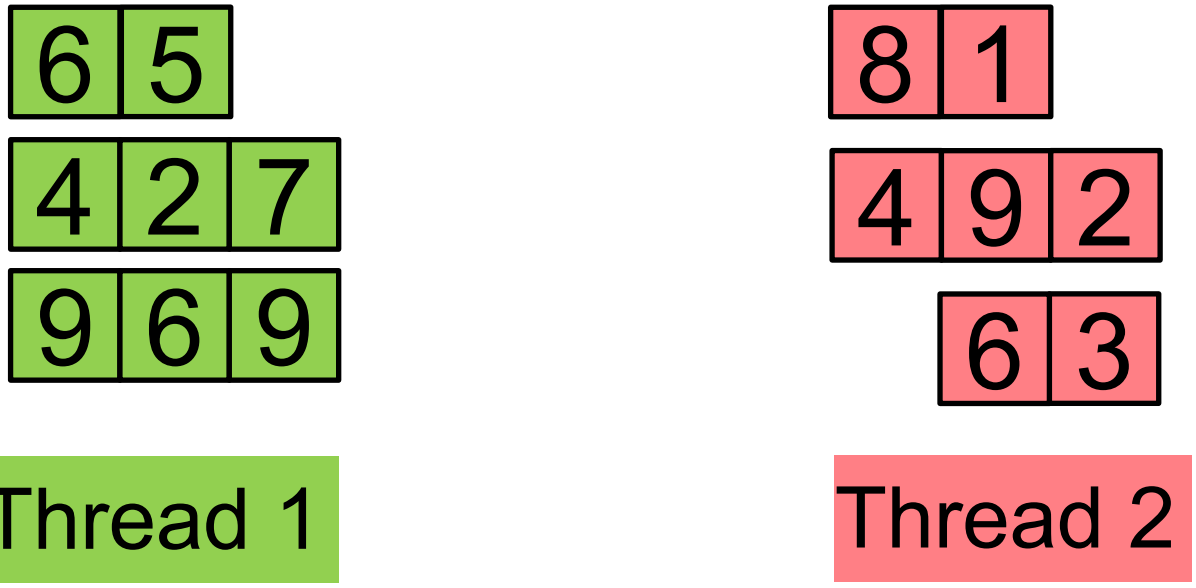
Probleme ”Embarrassingly parallel”

floor(N/P) elemente floor(15/2) = 7



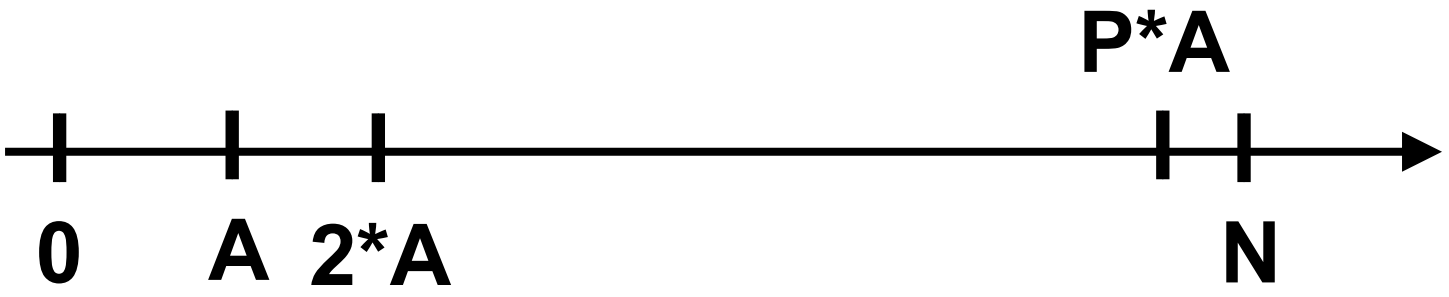
Probleme ”Embarrassingly parallel”

ceil(N/P) elemente **ceil(15/2) = 8**



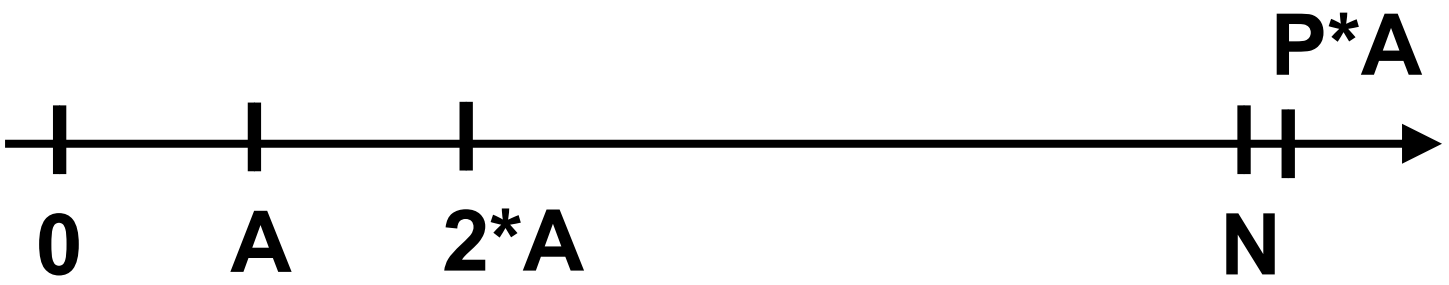
Probleme ”Embarrassingly parallel”

A = floor(N/P)



Probleme ”Embarrassingly parallel”

A = ceil(N/P)



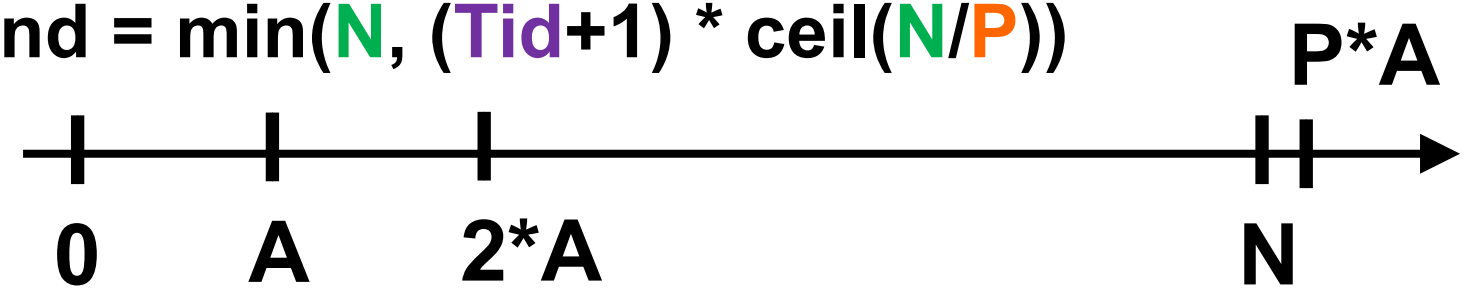
Probleme ”Embarrassingly parallel”

Formule elegante:

Tid este identificator de thread, are valori de la 0 la P-1

start = Tid * ceil(N/P)

end = min(N, (Tid+1) * ceil(N/P))



Măsurarea timpului de execuție

time ./executabil p a r a m e t r i

Măsurare timp – Linia de comandă

time *sleep 5*

real 0m5.001s
user 0m0.000s
sys 0m0.001s

time sleep 5

sleep 5 0.00s user 0.00s system 0% cpu 5.002 total

/usr/bin/time sleep 5

0.00user 0.00system 0:05.00elapsed 0%CPU (0avgtext+0avgdata 2076maxresident)k
0inputs+0outputs (0major+73minor)pagefaults 0swaps

Măsurare timp – Linia de comandă

***time** sleep 5*

```
real    0m5.001s
user    0m0.000s
sys     0m0.001s
```

Wall clock time – Timpul trecut de la pornirea programului – Pe acesta îl folosim

time sleep 5

```
sleep 5  0.00s user 0.00s system 0% cpu 5.002 total
```

/usr/bin/time sleep 5

```
0.00user 0.00system 0:05.00elapsed 0%CPU (0avgtext+0avgdata 2076maxresident)k
0inputs+0outputs (0major+73minor)pagefaults 0swaps
```

Măsurare timp – Linia de comandă

time sleep 2

real 0m2.0**21**s
user 0m0.000s
sys 0m0.000s

time sleep 2

real 0m2.0**18**s
user 0m0.000s
sys 0m0.016s

time sleep 2

real 0m2.0**16**s
user 0m0.000s
sys 0m0.000s

time sleep 2

real 0m2.0**15**s
user 0m0.000s
sys 0m0.000s

Timpii măsurați nu sunt exacți.

- Dacă avem timpi foarte mici nu sunt relevanți. De preferat timpi peste 1 secundă.
- Pentru a măsura corect trebuie să facem medie a timpilor după mai multe rulări.

Măsurare timp – Linia de comandă

***time** sleep 5*

real 0m5.001s
user 0m0.000s
sys 0m0.001s

Suma timpului petrecut în user space pe fiecare core.

time sleep 5

sleep 5 0.00s user 0.00s system 0% cpu 5.002 total

/usr/bin/time sleep 5

0.00user 0.00system 0:05.00elapsed 0%CPU (0avgtext+0avgdata 2076maxresident)k
0inputs+0outputs (0major+73minor)pagefaults 0swaps

Măsurare timp – Linia de comandă

Suma timpului petrecut în user space pe fiecare core.

```
time timeout 5 ./useAllCPU 12

real    0m4.075s
user    0m47.797s
sys     0m0.031s
```

Măsurare timp – Linia de comandă

***time** sleep 5*

real 0m5.001s
user 0m0.000s
sys 0m0.001s

Suma timpului petrecut în kernel pe fiecare core.

time sleep 5

sleep 5 0.00s user 0.00s system 0% cpu 5.002 total

/usr/bin/time sleep 5

0.00user 0.00system 0:05.00elapsed 0%CPU (0avgtext+0avgdata 2076maxresident)k
0inputs+0outputs (0major+73minor)pagefaults 0swaps

Măsurare timp – Linia de comandă

Orice I/O este făcut de Kernel

```
time dd if=/dev/zero of=file.txt count=1024 bs=1048576
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB) copied, 9.4847 s, 113 MB/s

real    0m9.490s
user    0m0.000s
sys     0m0.992s
```

Măsurare timp – Din program

```
clock_t t;
t = clock();
WORK();
t = clock() - t;
double time_taken = ((double)t)/CLOCKS_PER_SEC; // in seconds
```

Măsurare timp – Din program

DO NOT USE

```
clock_t t;
t = clock();
WORK();
t = clock() - t;
double time_taken = ((double)t)/CLOCKS_PER_SEC; // in seconds
```

Măsurare timp – Din program

DO NOT USE

```
clock_t t;
t = clock();
WORK();
t = clock() - t;
double time_taken = ((double)t)/CLOCKS_PER_SEC; // in seconds
```

Din **man**: the value returned by **clock**() also includes the times of any children.

Măsurare timp – Din program

```
struct timespec start, finish;
double elapsed;
clock_gettime(CLOCK_MONOTONIC, &start);
WORK();
clock_gettime(CLOCK_MONOTONIC, &finish);
elapsed = (finish.tv_sec - start.tv_sec);
elapsed += (finish.tv_nsec - start.tv_nsec) / 1000000000.0;
```

Măsurare timp cu sau fără I/O?

Măsuri

- T - Timpul total necesar execuției programului paralel
- P - Numărul de procesoare utilizate
- G – Timp execuție cel mai rapid algoritm secvențial
- S – Speedup

$$- S = \frac{G}{T}$$

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1			

$$S = \frac{6.14}{6.14}$$

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8		

$$S = \frac{6.14}{7.76}$$

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 0.78	

$$S = \frac{6.14}{7.82}$$

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 0.78	S = 0.78

$$S = \frac{6.14}{7.87}$$

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 0.78	S = 0.78

WRONG

$$S = \frac{6.14}{7.87}$$

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1			

$$S = \frac{6.15}{6.15}$$

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8		

$$S = \frac{6.15}{7.77}$$

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 1.55	

$$S = \frac{6.15}{3.95}$$

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 1.55	S = 3.05

$$S = \frac{6.15}{2.01}$$

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 1.55	S = 3.05

De ce nu **S = P** ?

Timpi adunare a doi vectori C = A + B

Nu se ține cont de timpul de citire/scriere RAM.

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 1.55	S = 3.05

De ce nu **S = P** ? S = O(P) este **ideal**.

Consistența?

- Demonstrație formală

- Stress test
  - mereu comparați cu rezultatul algoritmului secvențial

Workflow - Testarea programelor

